

STA_445_Assignment 7

Ivan Perisic

Load your packages here:

Problem 1:

The `infmort` data set from the package `faraway` gives the infant mortality rate for a variety of countries. The information is relatively out of date, but will be fun to graph. Visualize the data using by creating scatter plots of mortality vs income while faceting using `region` and setting color by `oil` export status. Utilize a $\log_{10}$ transformation for both `mortality` and `income` axes. This can be done either by doing the transformation inside the `aes()` command or by utilizing the `scale_x_log10()` or `scale_y_log10()` layers. The critical difference is if the scales are on the original vs log transformed scale. Experiment with both and see which you prefer.

- a. The `rownames()` of the table gives the country names and you should create a new column that contains the country names. `*rownames`

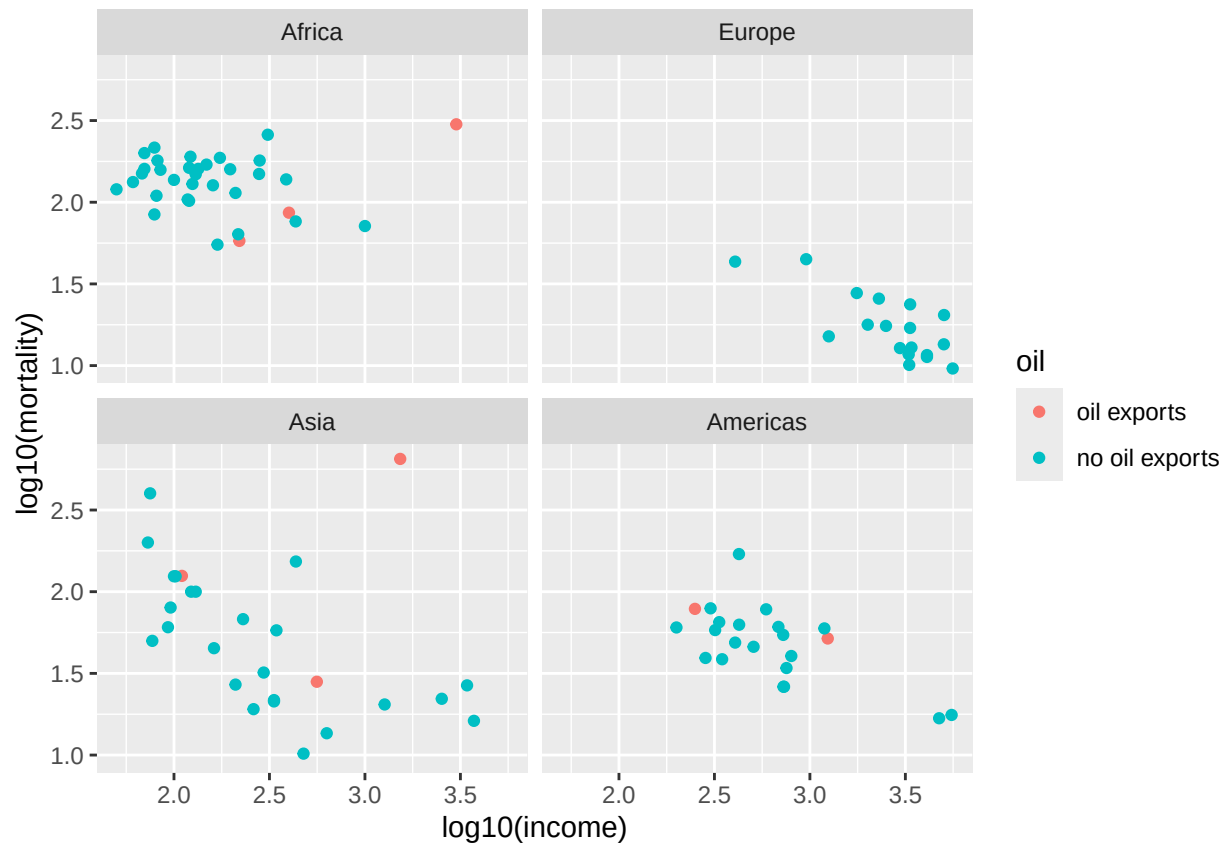
```
infmort_data <- infmort %>%  
  mutate(country = rownames(infmort))  
head(infmort_data)
```

```
##           region income mortality      oil  
## Australia      Asia   3426      26.7 no oil exports  
## Austria        Europe 3350      23.7 no oil exports  
## Belgium        Europe 3346      17.0 no oil exports  
## Canada      Americas 4751      16.8 no oil exports  
## Denmark        Europe 5029      13.5 no oil exports  
## Finland        Europe 3312      10.1 no oil exports  
##  
##           country  
## Australia      Australia  
## Austria        Austria  
## Belgium        Belgium  
## Canada          Canada  
## Denmark        Denmark  
## Finland        Finland
```

- b. Create scatter plots with the `log10()` transformation inside the `aes()` command.

```
ggplot(infmort_data, aes(x = log10(income), y = log10(mortality), color = oil)) +  
  geom_point() +  
  facet_wrap(~region)
```

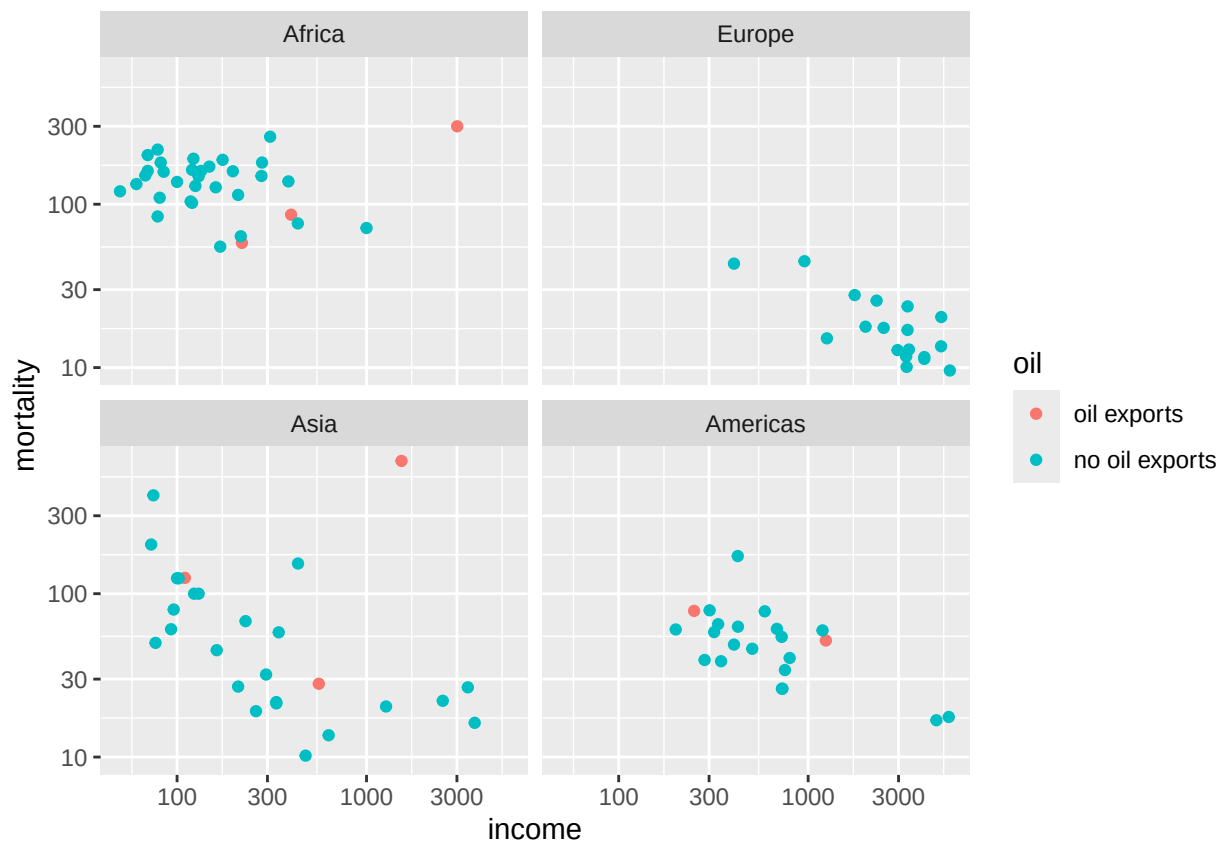
```
## Warning: Removed 4 rows containing missing values or values outside the scale range  
## (`geom_point()`).
```



c. Create the scatter plots using the `scale_x_log10()` and `scale_y_log10()`. Set the major and minor breaks to be useful and aesthetically pleasing. Comment on which version you find easier to read.

```
ggplot(infmort_data, aes(x = income, y = mortality, color = oil)) +
  geom_point() +
  facet_wrap(~region) +
  scale_x_log10() +
  scale_y_log10()
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



A more readable version using `scale_x_log10()` and `scale_y_log10()` does not change the values of the ticks, but applies the transformation, which is preferred over using `log10()` inside `aes()` arguments for the scale as in the previous example

- d. The package `ggrepel` contains functions `geom_text_repel()` and `geom_label_repel()` that mimic the basic `geom_text()` and `geom_label()` functions in `ggplot2`, but work to make sure the labels don't overlap. Select 10-15 countries to label and do so using the `geom_text_repel()` function.

```
plot_data <- infmort_data %>%
  filter(income > 0, mortality > 0)

label_data <- plot_data %>%
  sample_n(12)

ggplot(plot_data, aes(x = income, y = mortality, color = oil)) +
  geom_point() +
  facet_wrap(~region) +
  scale_x_log10() +
  scale_y_log10() +
  geom_text_repel(data = label_data, aes(label = country), size = 3)
```



I wanted to use the set seed so it would give me the same countries but I wasn't sure if that would be accepted. So I labeled 12 countries here and there shouldn't be any overlap I hope this is satisfactory.

Problem 2

Using the `datasets::trees` data, complete the following:

- Create a regression model for $y = \text{Volume}$ as a function of $x = \text{Height}$.

```
model <- lm(Volume ~ Height, data = trees)
```

- Using the `str(your model's name)` command, to get a list of all the information stored in the linear model object. Use `$` to extract the slope and intercept of the regression line (the coefficients).

```
str(model)
```

```
## List of 12
## $ coefficients : Named num [1:2] -87.12 1.54
##   .. attr(*, "names")= chr [1:2] "(Intercept)" "Height"
## $ residuals    : Named num [1:31] -10.6109 -2.8941 0.0926 -7.5976 -19.0877 ...
##   .. attr(*, "names")= chr [1:31] "1" "2" "3" "4" ...
## $ effects      : Named num [1:31] -167.98 53.86 2.05 -5.89 -17.64 ...
##   .. attr(*, "names")= chr [1:31] "(Intercept)" "Height" "" "" ...
## $ rank         : int 2
```

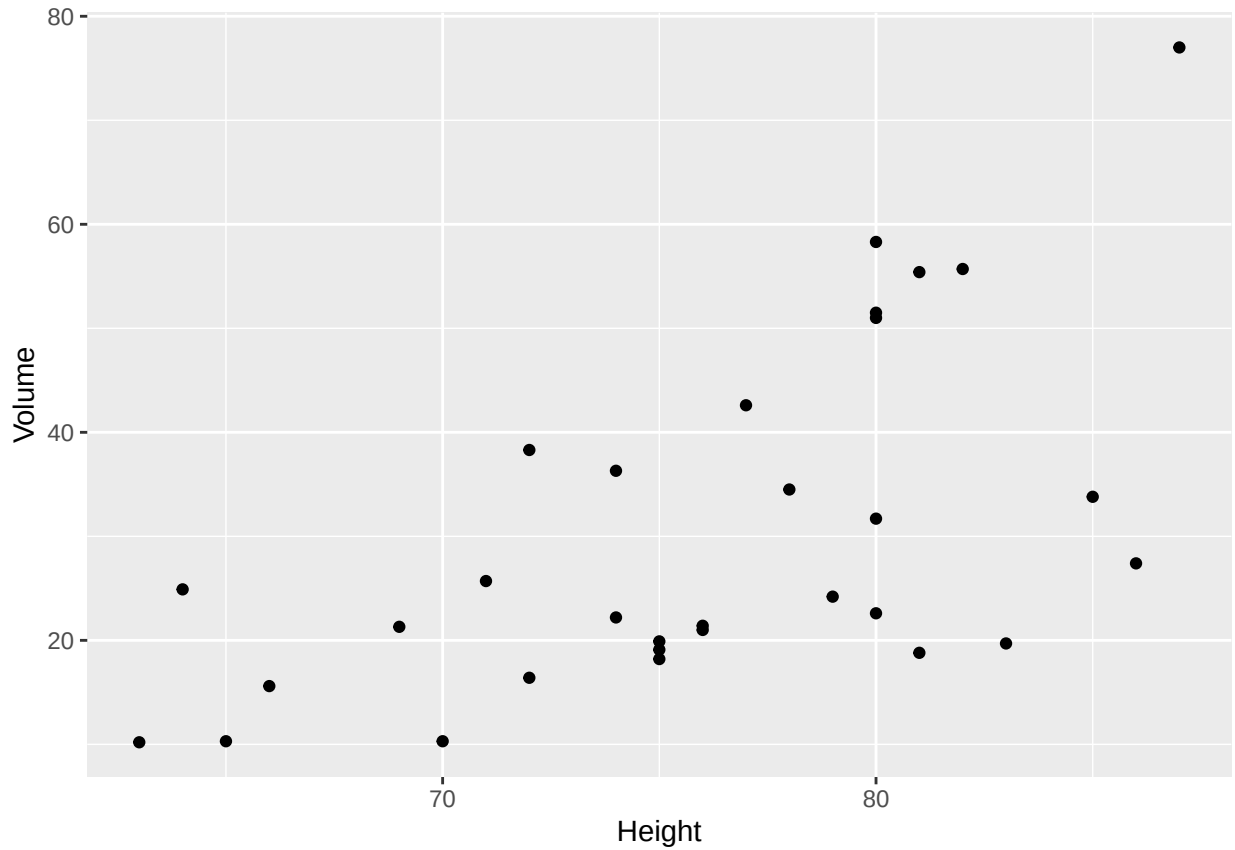
```
## $ fitted.values: Named num [1:31] 20.9 13.2 10.1 24 37.9 ...
## ..- attr(*, "names")= chr [1:31] "1" "2" "3" "4" ...
## $ assign      : int [1:2] 0 1
## $ qr          :List of 5
## ..$ qr       : num [1:31, 1:2] -5.57 0.18 0.18 0.18 0.18 ...
## .. ..- attr(*, "dimnames")=List of 2
## .. .. ..$ : chr [1:31] "1" "2" "3" "4" ...
## .. .. ..$ : chr [1:2] "(Intercept)" "Height"
## .. ..- attr(*, "assign")= int [1:2] 0 1
## ..$ qraux: num [1:2] 1.18 1.29
## ..$ pivot: int [1:2] 1 2
## ..$ tol   : num 1e-07
## ..$ rank  : int 2
## ..- attr(*, "class")= chr "qr"
## $ df.residual : int 29
## $ xlevels      : Named list()
## $ call         : language lm(formula = Volume ~ Height, data = trees)
## $ terms        :Classes 'terms', 'formula' language Volume ~ Height
## .. ..- attr(*, "variables")= language list(Volume, Height)
## .. ..- attr(*, "factors")= int [1:2, 1] 0 1
## .. .. ..- attr(*, "dimnames")=List of 2
## .. .. .. ..$ : chr [1:2] "Volume" "Height"
## .. .. .. ..$ : chr "Height"
## .. ..- attr(*, "term.labels")= chr "Height"
## .. ..- attr(*, "order")= int 1
## .. ..- attr(*, "intercept")= int 1
## .. ..- attr(*, "response")= int 1
## .. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
## .. ..- attr(*, "predvars")= language list(Volume, Height)
## .. ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
## .. .. ..- attr(*, "names")= chr [1:2] "Volume" "Height"
## $ model        :'data.frame': 31 obs. of 2 variables:
## ..$ Volume: num [1:31] 10.3 10.3 10.2 16.4 18.8 19.7 15.6 18.2 22.6 19.9 ...
## ..$ Height: num [1:31] 70 65 63 72 81 83 66 75 80 75 ...
## ..- attr(*, "terms")=Classes 'terms', 'formula' language Volume ~ Height
## .. .. ..- attr(*, "variables")= language list(Volume, Height)
## .. .. ..- attr(*, "factors")= int [1:2, 1] 0 1
## .. .. .. ..- attr(*, "dimnames")=List of 2
## .. .. .. .. ..$ : chr [1:2] "Volume" "Height"
## .. .. .. .. ..$ : chr "Height"
## .. .. ..- attr(*, "term.labels")= chr "Height"
## .. .. ..- attr(*, "order")= int 1
## .. .. ..- attr(*, "intercept")= int 1
## .. .. ..- attr(*, "response")= int 1
## .. .. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
## .. .. ..- attr(*, "predvars")= language list(Volume, Height)
## .. .. ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
## .. .. .. ..- attr(*, "names")= chr [1:2] "Volume" "Height"
## - attr(*, "class")= chr "lm"
```

```
model$coefficients
```

```
## (Intercept)      Height
##   -87.12361      1.54335
```

c. Using `ggplot2`, create a scatter plot of Volume vs Height.

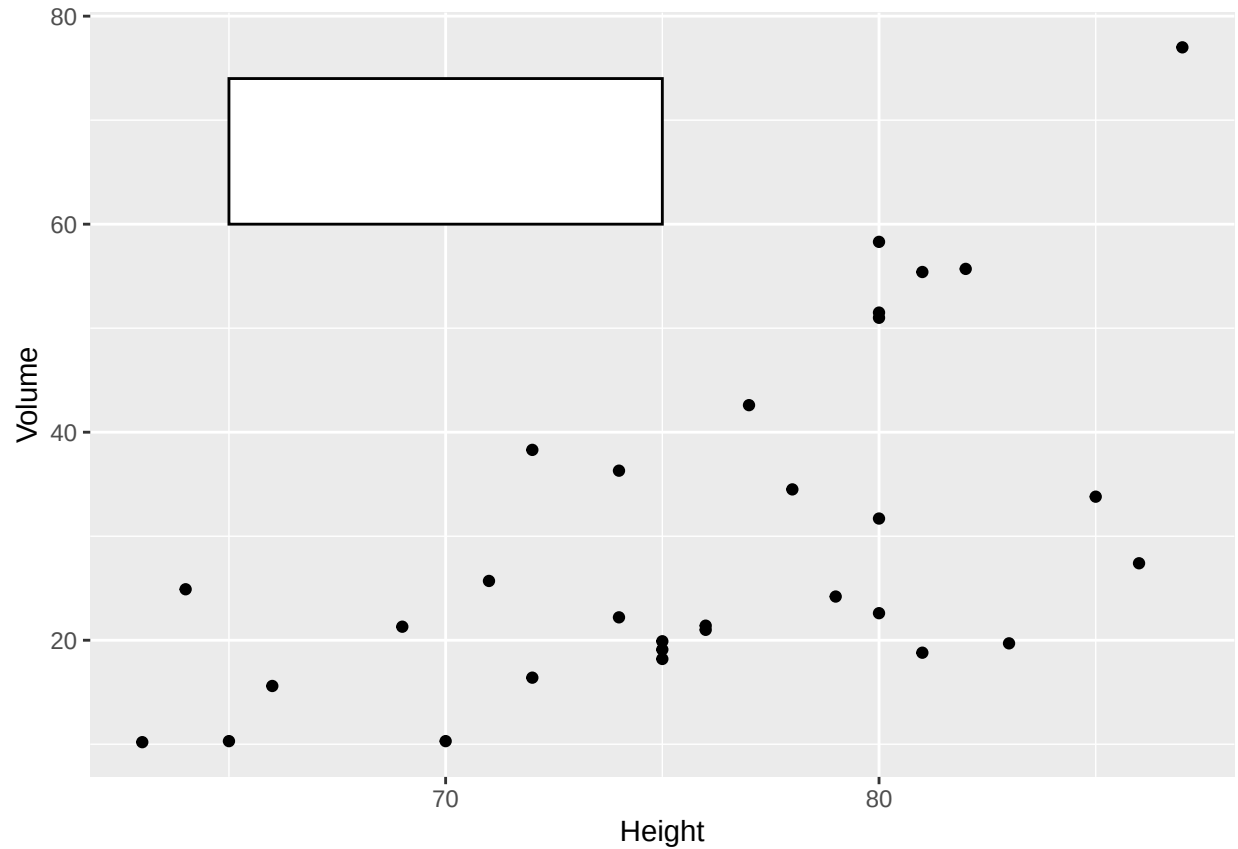
```
ggplot(trees, aes(x = Height, y = Volume)) +  
  geom_point()
```



d. Create a nice white filled rectangle to add text information to using by adding the following annotation layer.

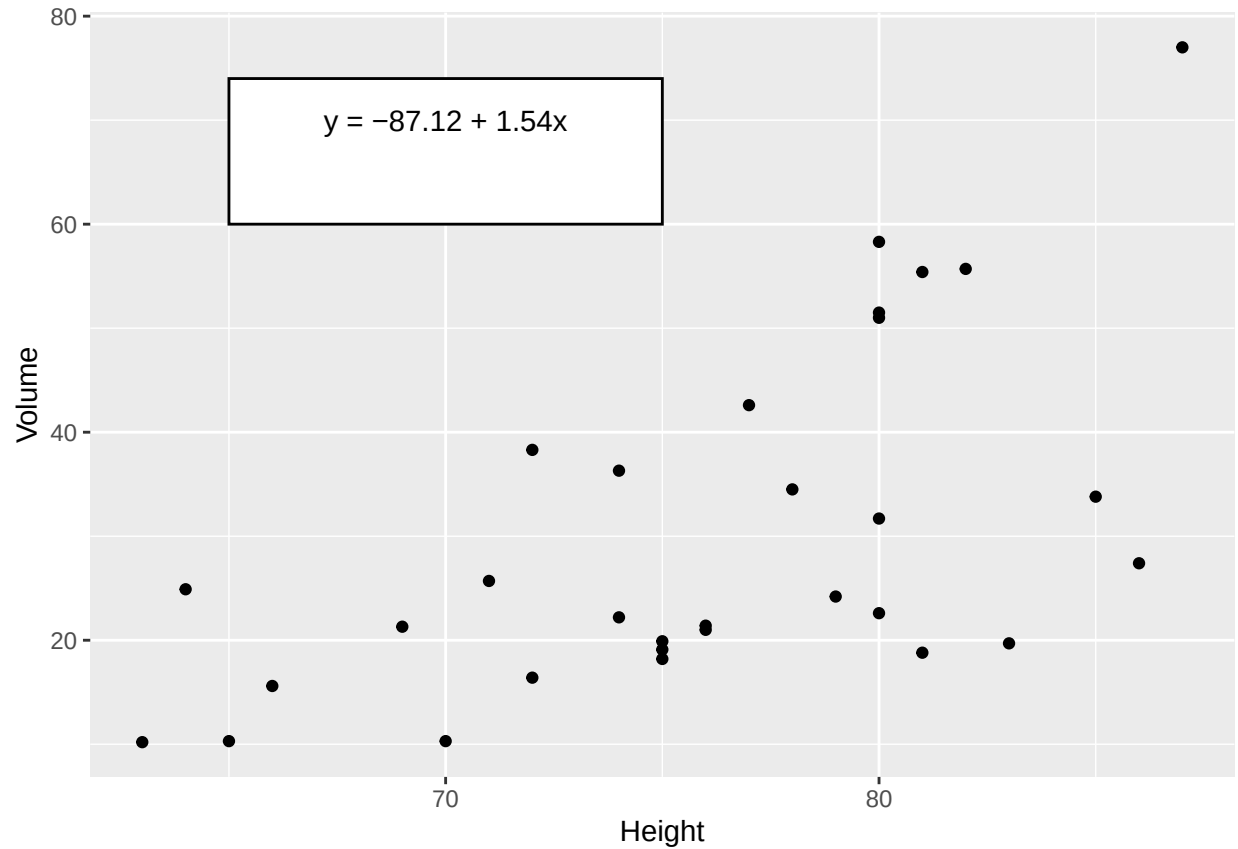
```
annotate('rect', xmin=65, xmax=75, ymin=60, ymax=74, fill='white', color='black')
```

```
ggplot(trees, aes(x = Height, y = Volume)) +  
  geom_point() +  
  annotate("rect", xmin = 65, xmax = 75, ymin = 60, ymax = 74,  
         fill = "white", color = "black")
```



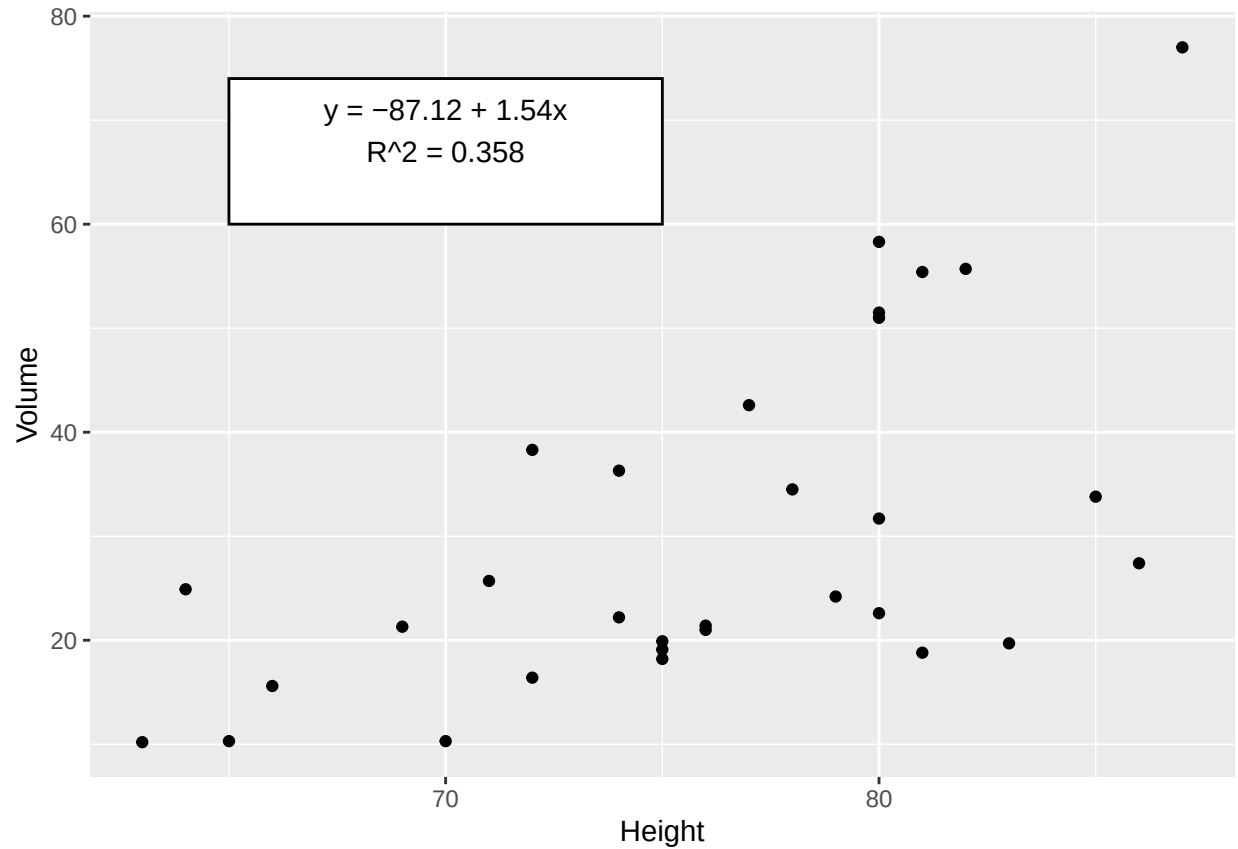
e. Add some annotation text to write the equation of the line $\hat{y}_i = -87.12 + 1.54x_i$ in the text area.

```
ggplot(trees, aes(x = Height, y = Volume)) +
  geom_point() +
  annotate("rect", xmin = 65, xmax = 75, ymin = 60, ymax = 74,
         fill = "white", color = "black") +
  annotate("text", x = 70, y = 70,
         label = "y = -87.12 + 1.54x")
```



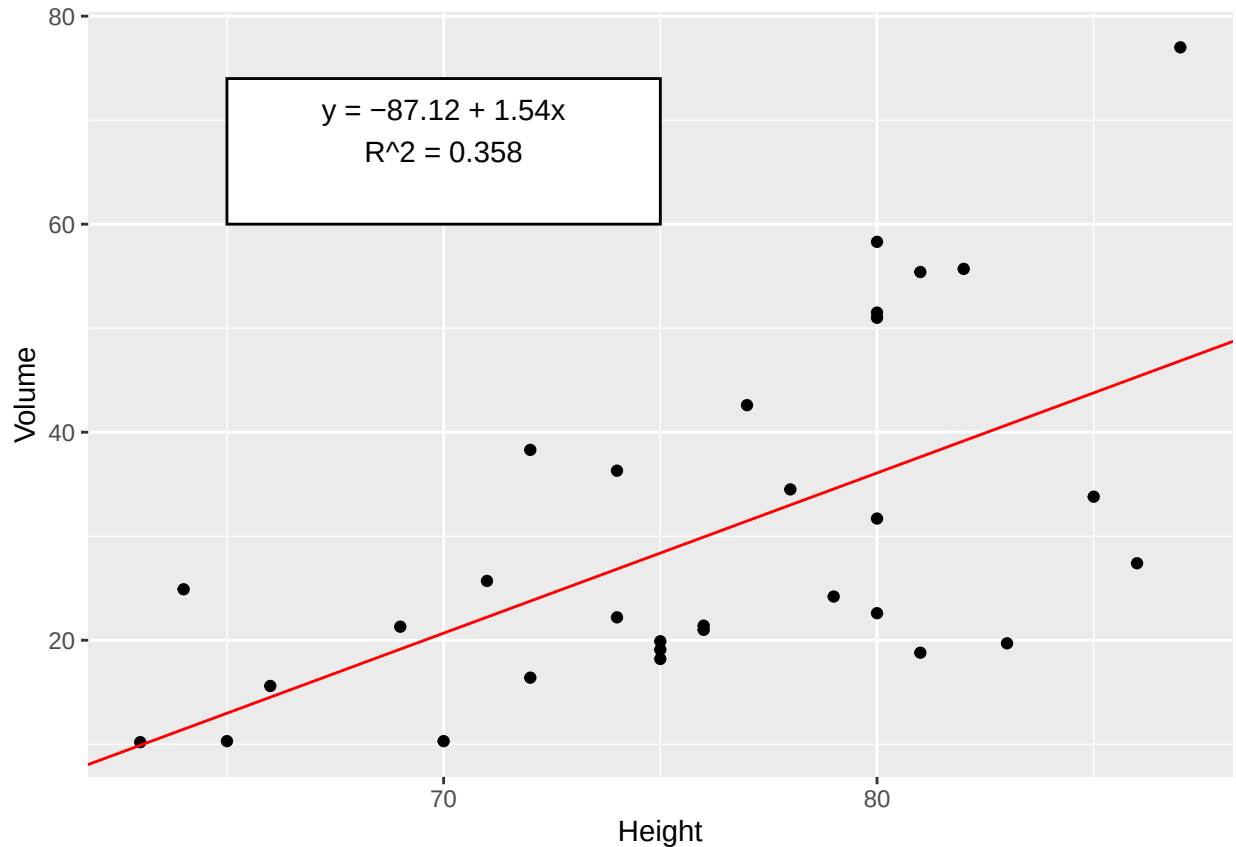
f. Add annotation to add $R^2 = 0.358$

```
ggplot(trees, aes(x = Height, y = Volume)) +
  geom_point() +
  annotate("rect", xmin = 65, xmax = 75, ymin = 60, ymax = 74,
         fill = "white", color = "black") +
  annotate("text", x = 70, y = 71,
         label = "y = -87.12 + 1.54x") +
  annotate("text", x = 70, y = 67,
         label = "R^2 = 0.358")
```



g. Add the regression line in red. The most convenient layer function to use is `geom_abline()`.

```
ggplot(trees, aes(x = Height, y = Volume)) +
  geom_point() +
  annotate("rect", xmin = 65, xmax = 75, ymin = 60, ymax = 74,
         fill = "white", color = "black") +
  annotate("text", x = 70, y = 71,
         label = "y = -87.12 + 1.54x") +
  annotate("text", x = 70, y = 67,
         label = "R^2 = 0.358") +
  geom_abline(intercept = -87.12, slope = 1.54, color = "red")
```



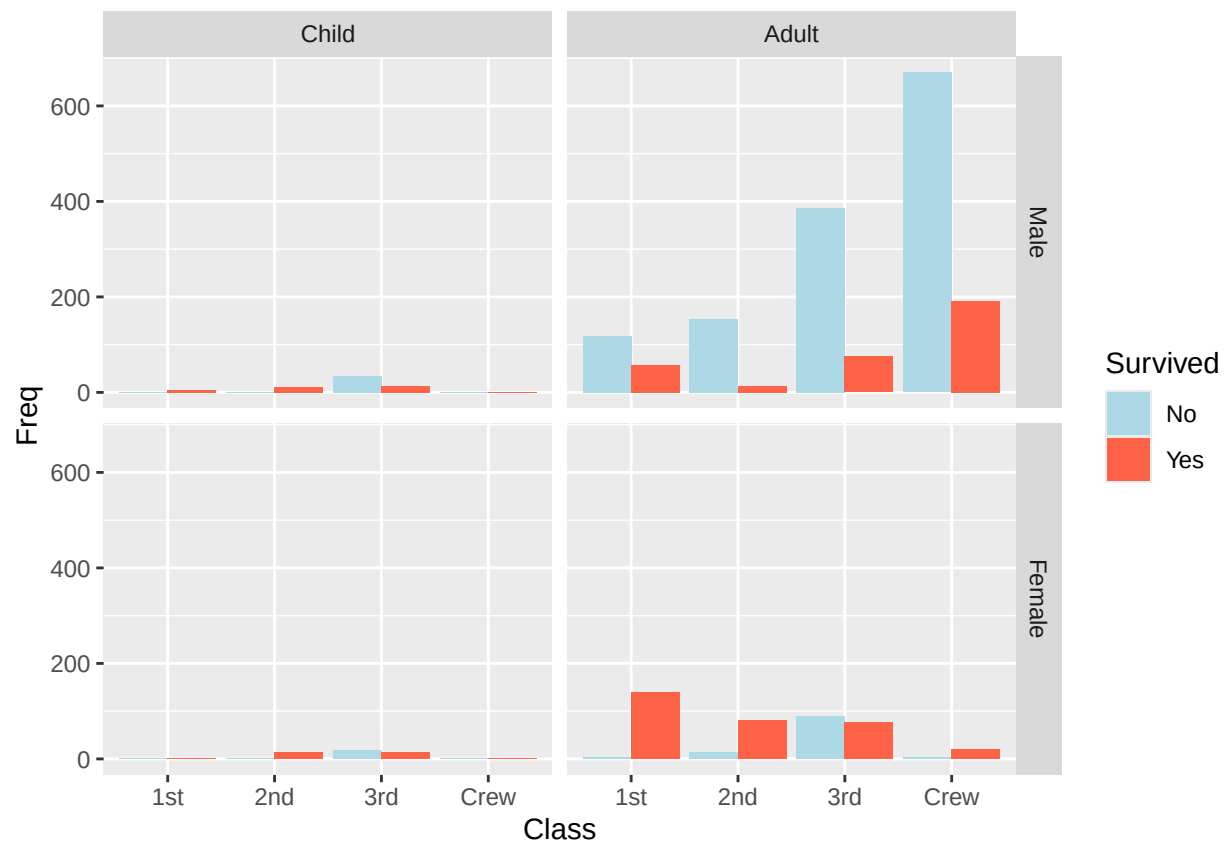
Problem 3

In `datasets::Titanic` table summarizes the survival of passengers aboard the ocean liner *Titanic*. It includes information about passenger class, sex, and age (adult or child). Create a bar graph showing the number of individuals that survived based on the passenger **Class**, **Sex**, and **Age** variable information. You'll need to use faceting and/or color to get all four variables on the same graph. Make sure that differences in survival among different classes of children are perceivable. *Unfortunately, the data is stored as a `table` and to expand it to a data frame, the following code can be used.*

```
Titanic <- Titanic %>% as.data.frame()
```

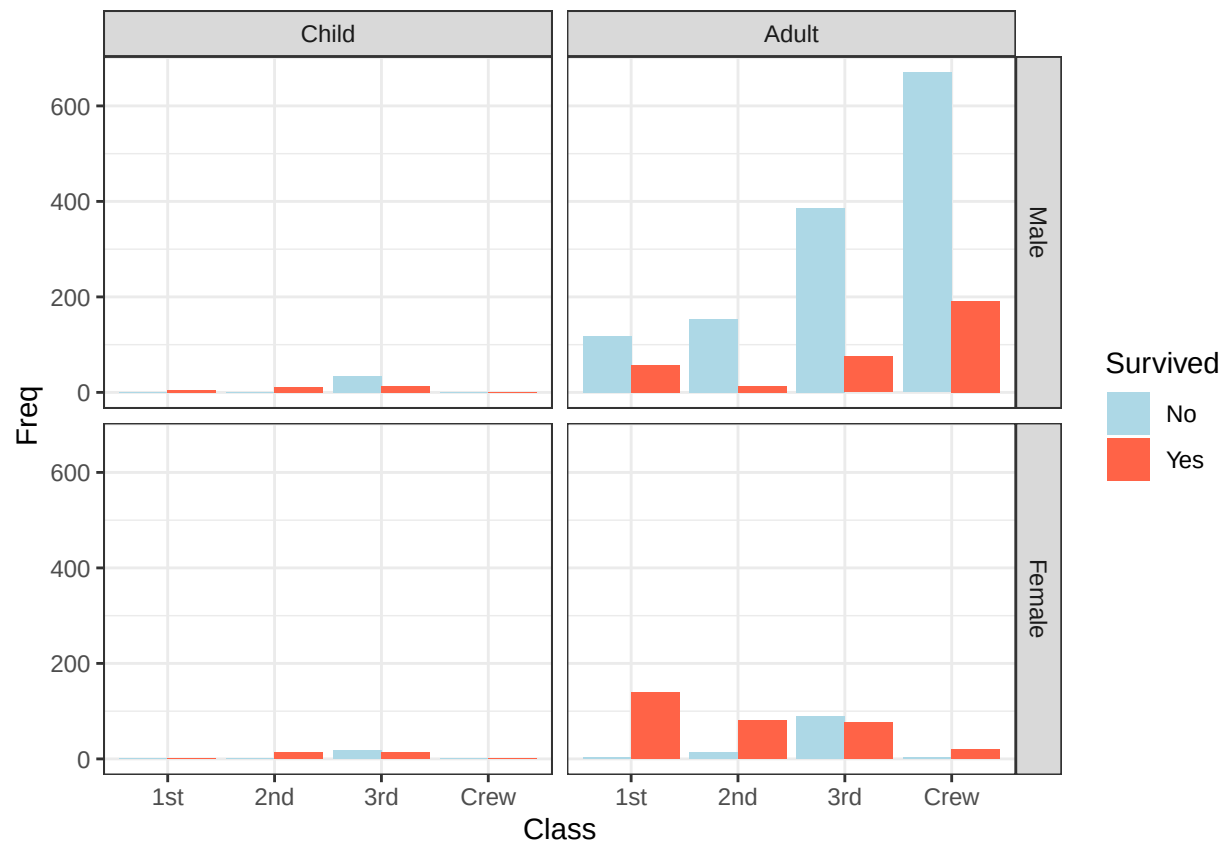
- Make this graph using the default theme. *If you use color to denote survivorship, modify the color scheme so that a cold color denotes death.*

```
ggplot(Titanic, aes(x = Class, y = Freq, fill = Survived)) +
  geom_col(position = "dodge") +
  facet_grid(Sex ~ Age) +
  scale_fill_manual(values = c("No" = "lightblue", "Yes" = "tomato"))
```



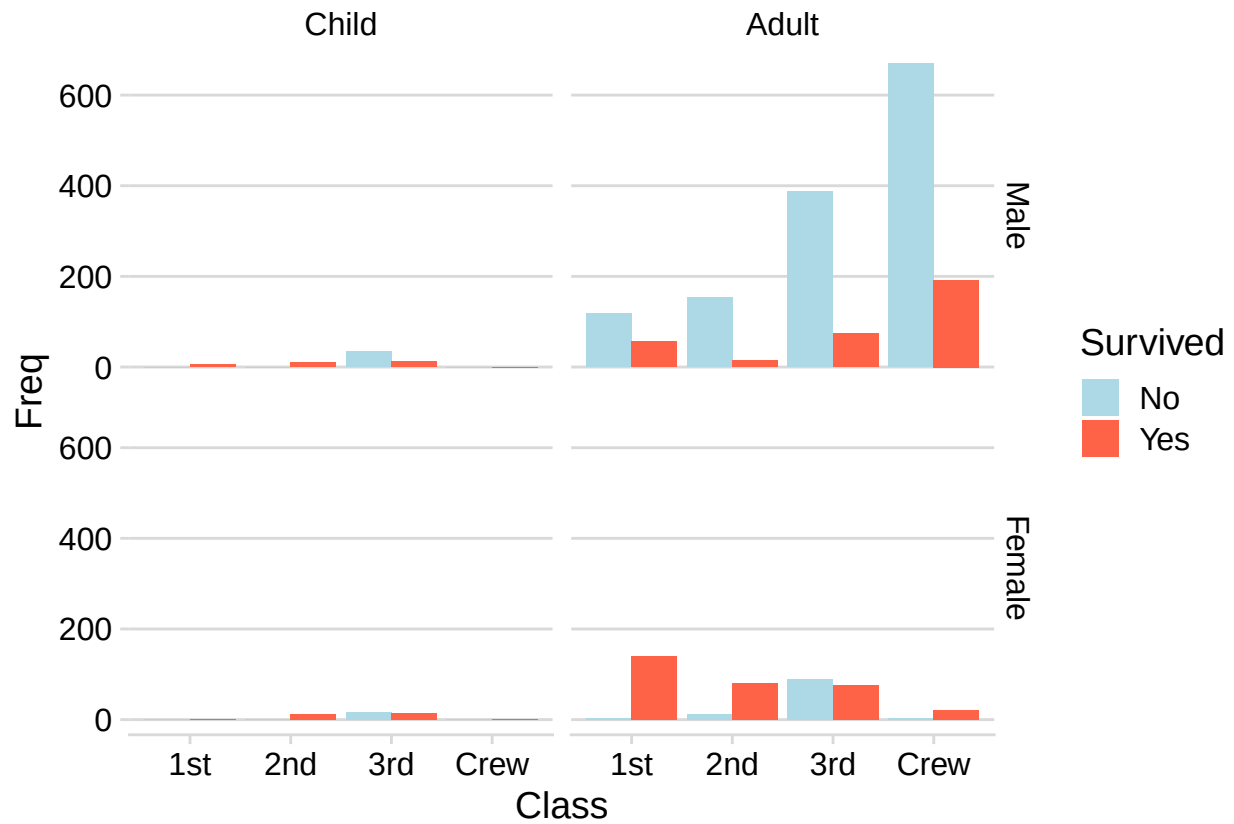
b. Make this graph using the `theme_bw()` theme.

```
ggplot(Titanic, aes(x = Class, y = Freq, fill = Survived)) +
  geom_col(position = "dodge") +
  facet_grid(Sex ~ Age) +
  scale_fill_manual(values = c("No" = "lightblue", "Yes" = "tomato")) +
  theme_bw()
```



c. Make this graph using the `cowplot::theme_minimal_hgrid()` theme.

```
ggplot(Titanic, aes(x = Class, y = Freq, fill = Survived)) +
  geom_col(position = "dodge") +
  facet_grid(Sex ~ Age) +
  scale_fill_manual(values = c("No" = "lightblue", "Yes" = "tomato")) +
  cowplot::theme_minimal_hgrid()
```



d. Why would it be beneficial to drop the vertical grid lines?

Vertical grid lines are not needed in this case because the x-axis is a categorical variable (the passenger class) and interpolation between x values is not useful. The plot with no vertical grid lines is easier to read.